

# PocketSmart AI

Coding, Output and Explanation

## 1. Source Code

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class BudgetRequest(BaseModel):
    income: float
    expenses: float
    goal: str

@app.post("/recommend")
def recommend(data: BudgetRequest):
    savings = data.income - data.expenses

    if savings >= data.income * 0.20:
        level = "Good"
        tip = "Keep building your emergency fund."

    elif savings >= data.income * 0.10:
        level = "Moderate"
        tip = "Try to increase your monthly savings gradually."

    else:
        level = "Needs Improvement"
        tip = "Review your flexible expenses."

    return {
        "savings": savings,
        "level": level,
        "goal": data.goal,
        "tip": tip
    }
```

## 2. Coding Explanation

### ***from fastapi import FastAPI***

Imports FastAPI, which is used to create the web application and API.

### ***from pydantic import BaseModel***

Imports BaseModel to define and validate the data received from the user.

### ***app = FastAPI()***

Creates the PocketSmart AI FastAPI application.

### ***BudgetRequest***

class BudgetRequest(BaseModel): income: float; expenses: float; goal: str. This defines the three inputs: income → user's monthly income; expenses → user's monthly expenses; goal → user's financial goal.

### ***/recommend***

@app.post("/recommend") creates an API endpoint that receives the user's budget information.

### ***Calculate savings***

savings = data.income - data.expenses. The application calculates Savings = Income – Expenses.

Example: ■30,000 – ■22,000 = ■8,000.

### ***Check saving level***

If savings are 20% or more of income, the level is Good. If savings are 10% or more, the level is Moderate. Otherwise, the level is Needs Improvement.

### ***Return result***

The return statement provides the savings amount, saving level, financial goal and recommendation.

### 3. Sample Input

Income: █30,000  
Expenses: █22,000  
Goal: Save money

#### **Calculation**

Savings = Income - Expenses

Savings = █30,000 - █22,000

Savings = █8,000

█8,000 is more than 10% of █30,000 but less than 20%, so the saving level is Moderate.

### 4. Expected Output

```
{  
  "savings": 8000,  
  "level": "Moderate",  
  "goal": "Save money",  
  "tip": "Try to increase your monthly savings gradually."  
}
```

#### **Simple Project Output**

```
=====
                PocketSmart AI
    Your Smart Budget & Recommendation
                Assistant
=====

Financial Goal : Save money
Monthly Savings: █8,000
Saving Level   : Moderate

AI Tip:
Try to increase your monthly savings gradually.
=====
```

## 5. How to Run

In the VS Code terminal:

```
pip install fastapi uvicorn pydantic
```

Then run:

```
uvicorn main:app --reload
```

Open the browser:

```
http://127.0.0.1:8000/docs
```

Use POST /recommend → Try it out to enter the sample data and see the JSON output.

## Final Result

PocketSmart AI calculates the user's savings from income and expenses and provides a simple saving level and financial recommendation.